
CODES CORRECTEURS D'ERREURS

pour l'UE HAI906I "Calcul formel avancé et applications"

POLYCOPIÉ - VERSION DU 08/10/2026

ROCCO MORA

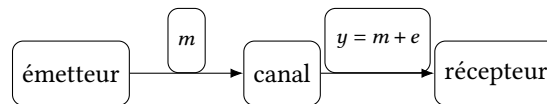
TABLE DES MATIÈRES

CHAPITRE 1	INTRODUCTION	PAGE 3
CHAPITRE 2	CODES (EN BLOCS) : NOTIONS DE BASE	PAGE 5
2.1	Alphabets, codes, distance, taux	5
2.2	Canaux de communication	8
2.3	Un code à chiffre de contrôle du monde réel : le numéro de sécurité sociale français	11
2.4	Premier aperçu de bons et de mauvais codes	12
2.5	Équivalence de codes	13
CHAPITRE 3	CODES LINÉAIRES	PAGE 14
3.1	Matrices génératrice et de contrôle de parité, dualité	14
3.2	Somme et intersection de codes	17
3.3	Poinçonnage et raccourcissement : fabriquer de nouveaux codes	18
3.4	Une galerie de familles classiques	20
3.5	Les codes non linéaires peuvent tout de même l'emporter	21
CHAPITRE A	CORPS FINIS	PAGE 22
A.1	Pourquoi ne pas simplement utiliser $\mathbb{Z}/n\mathbb{Z}$?	22
A.2	Construction de $\mathbb{F}_4$ à la main	22
A.3	La construction générale	23
A.4	Le groupe multiplicatif et les éléments primitifs	23
A.5	Inverses sans table : l'algorithme d'Euclide étendu	24
A.6	Sous-corps et morphisme de Frobenius	24
A.7	Table de référence rapide	24

Introduction

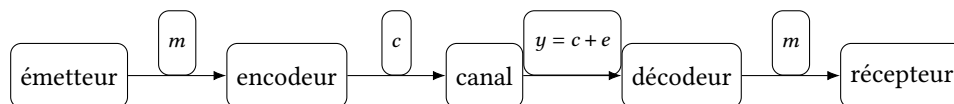
Le 20 août 1977, la sonde Voyager 2 quittait la Terre en emportant, parmi d'autres instruments, une caméra dont les images allaient devoir parcourir des milliards de kilomètres à travers le bruit interplanétaire avant d'atteindre une antenne radio au sol. Lorsque le signal arrivait, il était noyé dans un bruit thermique bien en deçà du niveau qu'une oreille ou un œil humain pouvait décoder directement, et pourtant les images de Jupiter, Saturne, Uranus et Neptune qui nous sont parvenues sont nettes. L'écart entre un canal bruité et un message fiable est comblé par les *codes correcteurs d'erreurs* : un filet de sécurité algébrique qui permet à un récepteur de reconstituer exactement ce qui a été envoyé, même si ce qui arrive réellement est corrompu.

Un exemple jouet. Avant tout formalisme, il est utile de voir l'idée centrale sur le plus petit exemple possible. Chaque fois que nous communiquons numériquement, nous le faisons à travers un *canal bruité* : ce que reçoit le récepteur n'est pas tout à fait ce qu'a envoyé l'émetteur.



Disons que l'émetteur souhaite transmettre le message binaire $m = (1, 0, 1)$, et que le canal inverse un bit, si bien que le récepteur reçoit $y = (0, 0, 1)$. En écrivant $y = m + e$, le *vecteur d'erreur* est ici $e = (1, 0, 0)$ — mais le récepteur ne voit jamais que y , jamais m ni e séparément, et n'a aucun moyen de savoir que quelque chose s'est mal passé : $(0, 0, 1)$ semble tout aussi plausible comme message que ne l'était $(1, 0, 1)$.

La solution de la théorie des codes n'est pas de modifier le canal — nous ne le pouvons pas — mais de modifier ce que nous envoyons. Nous *encodons* m au préalable, en ajoutant délibérément de la redondance, de sorte que les séquences transmises possibles (les *mots de code*) se trouvent loin les unes des autres.



Par exemple, au lieu d'envoyer directement $m = (1, 0, 1)$, répétons-le trois fois : $c = (1, 0, 1, 1, 0, 1, 1, 0, 1)$. Si la même erreur d'un seul bit se produit, le récepteur reçoit alors $y = (1, 0, 0, 1, 0, 1, 1, 0, 1)$ — et comme les huit mots de code possibles (un pour chaque choix de $m \in \{0, 1\}^3$) sont largement espacés les uns des autres, il existe exactement *un seul* mot de code à un bit inversé de y , à savoir c lui-même : l'erreur n'est pas seulement détectée mais *corrigée*, et m est retrouvé exactement, sans aucune ambiguïté.

Cet exemple jouet contient déjà tous les ingrédients auxquels le reste de ce cours consacrera de nombreux chapitres pour les préciser et les rendre efficaces : une notion de *distance* entre séquences, pour que « éloignés » et « mot de code le plus proche » aient un sens précis (Section 2.1); un *encodeur* transformant des messages courts en mots de code plus longs et bien séparés, et ce de façon systématique plutôt que par répétition ad hoc (Chapitre 3); et un *décodeur* qui, étant donné y , doit trouver efficacement le mot de

code le plus proche — trivial par force brute pour des messages de 3 bits, totalement infaisable par force brute dès que m compte un millier de bits (Chapitre 5) — le tout soumis à des compromis difficiles entre la quantité de redondance que nous pouvons nous permettre d'ajouter et le nombre d'erreurs que nous pouvons nous permettre de corriger (Chapitre 4).

La même idée, sous des habits différents, se retrouve dans chaque disque compact, chaque paquet Wi-Fi, chaque QR code sur une tasse de café, chaque baie RAID et chaque contrôleur SSD, et — plus surprenant encore — dans les schémas cryptographiques conçus pour résister aux ordinateurs quantiques, ainsi que dans les protocoles de partage de secret qui permettent d'ouvrir le coffre d'une banque seulement lorsque trois directeurs sur cinq sont d'accord.

Deux racines historiques. Le sujet trouve véritablement son origine dans le même bâtiment, aux Bell Telephone Laboratories, vers 1948. **Claude Shannon**, dans son article *A Mathematical Theory of Communication* (1948), montre, de façon purement existentielle et probabiliste, qu'un canal possède une *capacité* en deçà de laquelle une communication arbitrairement fiable est possible, et au-delà de laquelle elle ne l'est pas — mais l'argument de Shannon n'exhibe aucun code utilisable. **Richard Hamming**, qui travaillait dans le couloir voisin sur des calculateurs à relais s'arrêtant régulièrement à cause d'erreurs de parité détectées par des lecteurs de cartes perforées, posa la question complémentaire et constructive : étant donné qu'une erreur a été *détectée*, peut-elle aussi être *localisée et corrigée*? Son article de 1950, *Error Detecting and Error Correcting Codes*, répond par l'affirmative et donne la première famille non triviale de codes, encore utilisée aujourd'hui. Un troisième nom couramment associé à cette période initiale est celui de **Marcel Golay**, qui publia en 1949 (dans un article d'à peine une page) deux codes exceptionnels qui demeurent, soixante-quinze ans plus tard, parmi les objets combinatoires les plus remarquables connus.

Point de vue de ce cours. La théorie de Shannon porte sur la capacité *probabiliste*; la théorie que nous développons ici est *combinatoire* : au lieu des capacités de canal et des probabilités d'erreur, notre objet central est la **distance de Hamming** entre deux chaînes, et notre question centrale est de savoir combien de mots de code, deux à deux éloignés selon cette distance, peuvent être placés dans un espace fini. C'est ce point de vue combinatoire qui rend possibles des constructions explicites et des algorithmes de décodage explicites.

Jouons au jeu du théoricien des codes. Avant d'étudier rigoureusement la théorie des codes correcteurs d'erreurs, ses problèmes fondamentaux et les solutions trouvées au fil des décennies, il peut être intéressant d'essayer de construire nous-mêmes quelques codes. Par exemple :

1. Construire un code capable de toujours détecter deux erreurs.
2. Construire un code capable de toujours corriger deux erreurs.
3. Construire un code capable de toujours détecter une erreur en n'ajoutant qu'un seul bit de redondance.
4. Construire un code capable de toujours corriger une erreur en n'ajoutant qu'un seul bit de redondance.

Certaines de ces questions sont-elles plus faciles que d'autres ? Pensez-vous qu'elles sont toutes réalisables, ou que certaines sont théoriquement impossibles ?

Codes (en blocs) : notions de base

§ 2.1 Alphabets, codes, distance, taux

Definition 2.1. Soit A un ensemble fini (un *alphabet*) avec $|A| = q$, et soit $n \in \mathbb{N}^*$. Un *code de longueur n sur A* est simplement un sous-ensemble $C \subseteq A^n$. Ses éléments sont des *mots de code*. On note $M = |C|$ sa taille.

Aucune structure algébrique sur A ou C n'est supposée à ce stade — c'est délibéré : certains des meilleurs codes connus pour de petits paramètres ne sont pas linéaires, et il est important de voir le cadre général avant de se spécialiser.

Definition 2.2. Le *poids de Hamming* de $x \in A^n$ est

$$\text{wt}(x) := |\{i \in \{1, \dots, n\} \mid x_i \neq 0\}|.$$

Definition 2.3. La *distance de Hamming* entre $x, y \in A^n$ est

$$d(x, y) := |\{i \in \{1, \dots, n\} \mid x_i \neq y_i\}|.$$

On peut voir de façon équivalente la distance de Hamming entre $x, y \in A^n$ comme $\text{wt}(x - y)$.

Proposition 2.4. d est une *métrique* sur A^n , c'est-à-dire que

1. elle est *positive*, $d(x, y) \geq 0$ et $d(x, y) = 0$ si et seulement si $x = y$,
2. elle est *symétrique*, $d(x, y) = d(y, x)$,
3. et elle satisfait l'*inégalité triangulaire* $d(x, z) \leq d(x, y) + d(y, z)$.

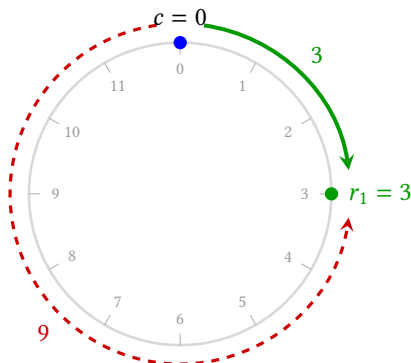
Exercice 2.5. Démontrer la Proposition 2.4.

Remark 2.6 (D'autres métriques existent). La distance de Hamming est la métrique que ce cours utilise presque exclusivement, mais ce n'est pas la seule qui présente un intérêt. Lorsque l'alphabet lui-même porte une structure cyclique, en « cadran d'horloge » — par exemple $A = \mathbb{Z}_q = \{0, 1, \dots, q-1\}$, par opposition à un simple ensemble fini non structuré — la *distance de Lee*

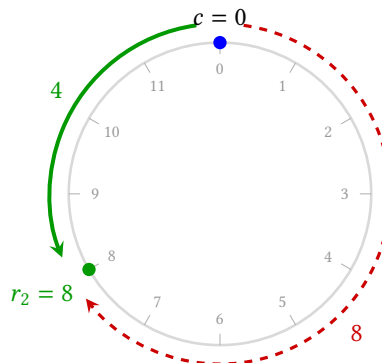
$$d_L(x, y) := \sum_{i=1}^n \min(|x_i - y_i|, q - |x_i - y_i|)$$

est souvent celle qui a un sens physique : elle mesure l'écart de chaque coordonnée comme le plus court

chemin autour d'un cycle de longueur q , plutôt que comme un simple indicateur de correspondance/non-correspondance, ce qui modélise les erreurs de type « phase » (par exemple dans certains schémas de modulation numérique) bien plus fidèlement que ne le fait la distance de Hamming. Voir l'Exercice 2.7 ci-dessous pour une preuve que d_L est bien une métrique, et pour sa relation avec d .

Exemple 1 dans $\mathbb{Z}_{12}$ 

$$d_L(0, 3) = \min(3, 9) = 3$$

Exemple 2 dans $\mathbb{Z}_{12}$ 

$$d_L(0, 8) = \min(8, 4) = 4$$

Un autre exemple est donné par la métrique de rang : dans ce cas, les mots de code ne sont pas des vecteurs mais des matrices, le poids est calculé comme le rang de la matrice et la distance comme le rang de la différence.

Exercice 2.7 (La métrique de Lee). Soit $A = \mathbb{Z}_q$ et rappelons la distance de Lee $d_L(x, y) := \sum_{i=1}^n \min(|x_i - y_i|, q - |x_i - y_i|)$ de la Remarque 2.6.

- Montrer que d_L est une métrique sur $\mathbb{Z}_q^n$: positivité, symétrie, $d_L(x, y) = 0 \iff x = y$, et l'inégalité triangulaire. (Pour l'inégalité triangulaire, commencer par prouver le cas coordonnée par coordonnée, $n = 1$; puis sommer sur les coordonnées.)
- Montrer que $d_L(x, y) \geq d(x, y)$ toujours, avec égalité lorsque $q \in \{2, 3\}$, et donner un exemple explicite avec $q \geq 4$ où l'inégalité est stricte.

Définition 2.8. La *distance minimale* d'un code C ayant au moins deux mots de code est

$$d(C) := \min_{\substack{x, y \in C \\ x \neq y}} d(x, y).$$

On note les paramètres de C par $(n, M, d)_q$, ou simplement (n, M, d) lorsque q est clair. Le *taux* (d'information) de C est $R := \log_q(M)/n \in [0, 1]$, et sa *distance relative* est $\delta := d/n \in [0, 1]$.

Exercice 2.9 (L'inclusion contrôle la distance). Soient $C_1 \subseteq C_2 \subseteq A^n$ deux codes, chacun ayant au moins deux mots de code.

- Montrer que $d(C_1) \geq d(C_2)$.
- Donner un exemple explicite, avec $C_1 \subsetneq C_2 \subseteq \mathbb{F}_2^4$ de tailles toutes deux ≥ 2 , où l'inégalité est stricte ($d(C_1) > d(C_2)$), ainsi qu'un autre exemple où il y a égalité.

Il existe une tension entre R et δ : intuitivement, écarter M mots de code de sorte qu'ils soient deux à deux très éloignés (grand δ) laisse moins de « place » pour en placer beaucoup (grand R). Cette tension est le mieux capturée par une unique quantité combinatoire, qu'il vaut la peine d'isoler dès maintenant puisqu'elle organisera pour ainsi dire tout ce qui suit.

Définition 2.10 (Le problème combinatoire central). Pour des entiers $n, d \geq 1$ et $q \geq 2$, on définit

$$A_q(n, d) := \max\{|C| \mid C \subseteq A^n, |A| = q, d(C) \geq d\},$$

la plus grande taille possible pour *n'importe quel* code (linéaire ou non) de longueur n sur un alphabet de taille q garantissant une distance minimale au moins d . Un code $C \subseteq A^n$ avec $d(C) \geq d$ atteignant $|C| = A_q(n, d)$ est dit $A_q(n, d)$ -*optimal* (ou simplement *optimal* pour ces paramètres, lorsque q, n, d sont clairs d'après le contexte).

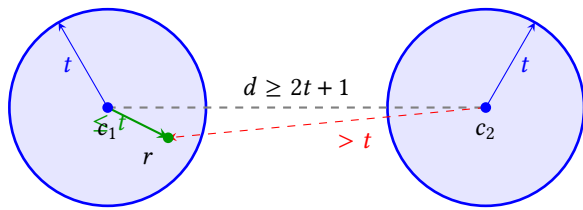
Déterminer $A_q(n, d)$ exactement, pour tous n, d, q , est sans doute le problème combinatoire central de tout ce cours : c'est précisément la question « à quel point un code peut-il être bon ? », dépouillée de toute considération hormis la longueur, la taille de l'alphabet et la distance minimale garantie. Le Chapitre 4 développe plusieurs bornes générales sur $A_q(n, d)$ (Singleton, Hamming, Plotkin, Gilbert–Varshamov), chacune capturant une obstruction combinatoire ou un argument d'existence différent ; pour ainsi dire chaque famille spécifique de codes construite plus loin dans ce cours est, au fond, une tentative de cerner exactement $A_q(n, d)$ pour une certaine famille infinie de paramètres — ou de s'en approcher utilement avec une construction qui se trouve aussi être efficacement *encodable et décodable*, ce dont $A_q(n, d)$ seul ne dit rien. Même pour $q = 2$ et des n assez modestes, la valeur exacte de $A_2(n, d)$ reste inconnue pour une infinité de paires (n, d) : après soixante-quinze ans, il s'agit toujours d'un domaine ouvert de la combinatoire énumérative, dont les records actuels sont répertoriés en ligne (<https://www.codetables.de/>). Elles ne suivent que les codes *linéaires* — il n'existe pas de table publique aussi complète pour la quantité pleinement générale, pas nécessairement linéaire, $A_q(n, d)$ de la Définition 2.10, précisément parce que, comme nous allons voir, abandonner l'exigence de linéarité rend l'espace de recherche, et donc le suivi de ce qui est actuellement connu, beaucoup plus vaste. Elles ne suivent que les codes *linéaires* — il n'existe pas de table publique aussi complète pour la quantité pleinement générale, pas nécessairement linéaire, $A_q(n, d)$ de la Définition 2.10, précisément parce qu'abandonner l'exigence de linéarité rend l'espace de recherche, et donc le suivi de ce qui est actuellement connu, beaucoup plus vaste. On peut néanmoins trouver certains résultats concernant les petits codes binaires non linéaires, par exemple à l'adresse <https://aeb.win.tue.nl/codes/binary-1.html>.

Exercice 2.11. Soit C un code $A_2(n, d)$ -optimal (Définition 2.10) avec d impair. Montrer qu'ajouter un bit de parité global à chaque mot de code (*extension*) transforme C en un code de longueur $n + 1$ et de distance minimale $d + 1$, de même taille. En déduire que $A_2(n + 1, d + 1) \geq A_2(n, d)$ pour tout d impair.

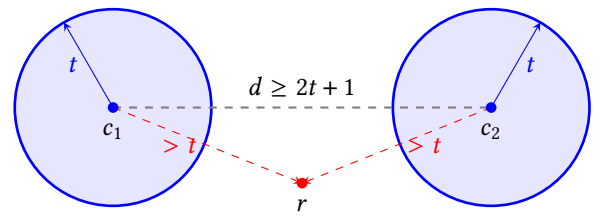
Proposition 2.12 (Distance et correction d'erreurs). Si $d(C) \geq 2t + 1$, alors pour tout $c \in C$ et tout $y \in A^n$ avec $d(y, c) \leq t$, le mot de code c est l'unique élément de C à distance au plus t de y . Par conséquent, un code de distance minimale d peut corriger tout motif de $t = \lfloor (d - 1)/2 \rfloor$ erreurs, et peut détecter (sans nécessairement les corriger) tout motif d'au plus $d - 1$ erreurs.

Démonstration. Si $c' \in C$, $c' \neq c$, vérifiait aussi $d(y, c') \leq t$, l'inégalité triangulaire donnerait $d(c, c') \leq 2t < d(C)$, ce qui contredit la définition de $d(C)$. ■

1 solution



0 solutions



Exercice 2.13. Soit $A = \{0, 1\}$ ($q = 2$) et considérons le code $C = \{(1, 1, 0, 1, 0), (0, 0, 0, 1, 1), (0, 1, 1, 0, 1)\} \subseteq A^5$.

- Calculer les trois poids de Hamming et les trois distances de Hamming deux à deux entre les trois mots de code.
- En déduire $d(C)$, et indiquer le plus grand t pour lequel C est garanti de détecter/corriger t erreurs quelconques.
- $y = (0, 1, 1, 1, 0)$ est-il à distance t d'un unique mot de code ? Si oui, lequel ?

Exercice 2.14. Soit $A = \{0, 1, 2\}$ ($q = 3$) et considérons le code $C = \{(0, 0, 1, 2), (1, 2, 0, 0), (2, 1, 2, 1), (0, 1, 1, 0)\} \subseteq A^4$.

- Calculer les quatre poids de Hamming et les six distances de Hamming deux à deux entre les quatre mots de code.
- En déduire $d(C)$, et indiquer le plus grand t pour lequel C est garanti de détecter/corriger t erreurs quelconques.
- $y = (0, 1, 2, 0)$ est-il à distance t d'un unique mot de code ? Si oui, lequel ?

Définition 2.15. La *boule* de rayon t autour de $x \in A^n$ est $B(x, t) := \{z \in A^n \mid d(x, z) \leq t\}$, la *sphère* de rayon t autour de $x \in A^n$ est $S(x, t) := \{z \in A^n \mid d(x, z) = t\}$.

La Proposition 2.12 dit exactement que les boules $B(c, \lfloor (d-1)/2 \rfloor)$, $c \in C$, sont deux à deux disjointes — une image d'*empilement de sphères* qui réapparaîtra plus loin sous le nom de borne de Hamming.

Proposition 2.16 (Cardinaux des boules et des sphères). Pour tout $x \in A^n$ et $0 \leq t \leq n$, $|S(x, t)| = \binom{n}{t}(q-1)^t$ et $|B(x, t)| = \sum_{i=0}^t \binom{n}{i}(q-1)^i$.

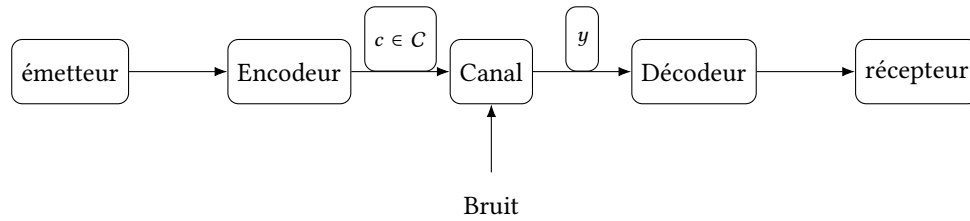
Exercice 2.17. Démontrer la Proposition 2.16.

Exercice 2.18. Sur $A = \{0, 1\}$, $n = 6$, calculer $|B(x, 2)|$ (le nombre de chaînes binaires de longueur 6 à distance de Hamming au plus 2 d'une chaîne fixée x). Recommencer pour $q = 4$, rayon $t = 3$, en utilisant la formule générale pour la taille des boules. Comparer chaque réponse à q^n et commenter la rapidité avec laquelle les boules remplissent l'espace ambiant lorsque t croît.

§ 2.2 Canaux de communication

D'où viennent physiquement l'espace ambiant A^n et toute l'idée de corriger jusqu'à t erreurs ? Cette section précise, et généralise, l'image jouet du Chapitre 1 : là, un message m devenait un mot de code c , le canal ajoutait une erreur inconnue e , et le récepteur ne voyait que $y = c + e$. Nous fixons maintenant

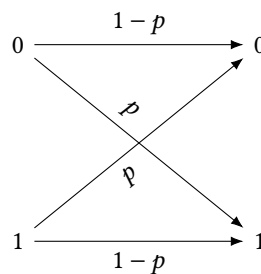
les modèles de bruit spécifiques auxquels ce cours se réfère chaque fois que nous demandons combien d'erreurs — ou combien de symboles *effacés* — un décodeur doit être construit pour tolérer.



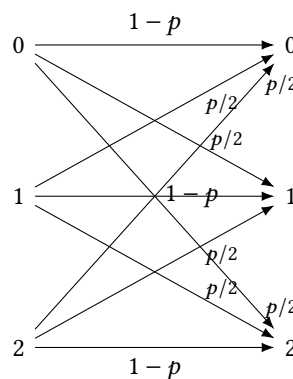
Définition 2.19. Un *canal discret* d'alphabet d'entrée A et d'alphabet de sortie B est spécifié par des probabilités de transition $P(y | x)$, $x \in A$, $y \in B$, avec $\sum_{y \in B} P(y | x) = 1$ pour tout x . Le canal est dit *sans mémoire* si, sur une séquence d'entrée $x = (x_1, \dots, x_n) \in A^n$, chaque symbole de sortie est produit indépendamment : $\Pr[y | x] = \prod_{i=1}^n P(y_i | x_i)$.

Tout canal utilisé dans ce cours est un canal discret sans mémoire (CDSM, ou DMC en anglais). Les canaux réels sont rarement *exactement* sans mémoire, mais le modèle sans mémoire est la bonne première approximation, et c'est celui par rapport auquel le théorème de capacité de Shannon et la plupart des constructions de ce cours sont énoncés.

Définition 2.20 (Canal binaire symétrique). Le *canal binaire symétrique* $\text{BSC}(p)$, $0 \leq p \leq \frac{1}{2}$, a $A = B = \{0, 1\}$ et inverse le bit transmis indépendamment avec probabilité p : $P(1 | 0) = P(0 | 1) = p$, $P(0 | 0) = P(1 | 1) = 1 - p$.



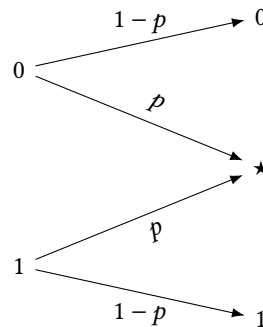
Définition 2.21 (Canal symétrique q -aire). Le *canal symétrique q -aire* $\text{QSC}_q(p)$ a $A = B = \{0, \dots, q-1\}$, transmet chaque symbole correctement avec probabilité $1 - p$, et sinon le remplace par *chacun* des $q-1$ autres symboles avec probabilité égale $p/(q-1)$: $P(x | x) = 1 - p$, $P(y | x) = p/(q-1)$ pour $y \neq x$.



(Représenté pour $q = 3$, où chaque flèche hors diagonale porte la probabilité $p/(q-1) = p/2$; en général, chaque entrée possède $q-1$ flèches hors diagonale, chacune de probabilité $p/(q-1)$.)

Les erreurs ne sont pas le seul type de corruption qu'il vaille la peine de modéliser séparément : parfois le récepteur sait exactement *quels* symboles ont été perdus (un secteur de disque qui n'a pas pu être lu, un paquet réseau qui n'est jamais arrivé) sans savoir ce qu'ils disaient — une situation qualitativement plus facile à gérer qu'une erreur en position inconnue, et suffisamment fréquente pour mériter son propre modèle de canal.

Définition 2.22 (Canal binaire à effacement). Le *canal binaire à effacement* $\text{BEC}(p)$ a pour alphabet d'entrée $\{0, 1\}$ et pour alphabet de sortie $\{0, 1, \star\}$, où $\star$ (l'*effacement*) signifie « perdu, position connue » : chaque bit est transmis correctement avec probabilité $1 - p$ et effacé avec probabilité p — il n'est *jamais* inversé vers la mauvaise valeur.



Définition 2.23 (Canal q -aire à effacement). Le *canal q -aire à effacement* généralise ceci à l'alphabet d'entrée $\{0, \dots, q - 1\}$ et à l'alphabet de sortie $\{0, \dots, q - 1, \star\}$: chaque symbole est transmis correctement avec probabilité $1 - p$ et effacé avec probabilité p .

Exercice 2.24. Représenter graphiquement le canal q -aire à effacement.

Enfin, nous précisons la règle de décodage que ce cours a déjà utilisée de façon informelle, et que nous étudierons algorithmiquement dans toute sa généralité plus avant.

Définition 2.25 (Décodage au plus proche voisin). Étant donné un mot reçu $y \in B^n$, le *décodage au plus proche voisin* produit

$$\hat{c}_{\text{NN}} := \arg \min_{c \in C} d(y, c)$$

(en départageant arbitrairement les ex æquo), c'est-à-dire un mot de code le plus proche de y pour la distance de Hamming. Il s'agit d'une règle de décision purement géométrique indépendante des caractéristiques probabilistes du canal.

Définition 2.26 (Décodage au maximum de vraisemblance). Étant donné un mot reçu $y \in B^n$, le *décodage au maximum de vraisemblance* (ML, pour *Maximum Likelihood*) produit

$$\hat{c}_{\text{ML}} := \arg \max_{c \in C} \Pr[y \mid c]$$

(en départageant arbitrairement les ex æquo), c'est-à-dire un mot de code qui maximise la probabilité d'observer la séquence y sachant que c a été transmis.

Exercice 2.27 (Rapprochement des décodages sur un canal sans mémoire $\text{QSC}_3(p)$). Soit un canal symétrique 3-aire $\text{QSC}_3(p)$ de paramètre d'erreur $p < 2/3$. Considérons un code $C \subseteq \{0, 1, 2\}^n$ et un vecteur reçu $y \in \{0, 1, 2\}^n$.

- (a) Expression de la vraisemblance : Pour un mot de code $c \in C$ situé à distance de Hamming $d = d(y, c)$ de y , exprimer la probabilité de transition $\Pr[y | c]$ en fonction de n , d et p .
- (b) Équivalence des règles : Montrer que la condition $p < 2/3$ implique que la fonction $d \mapsto \Pr[y | c]$ est strictement décroissante en fonction de d . En déduire que les règles de décodage au plus proche voisin ($\hat{c}_{NN}$) et au maximum de vraisemblance ($\hat{c}_{ML}$) mènent aux mêmes choix de mots de code.

Exercice 2.28 (Divergence des règles de décodage sur un canal avec mémoire). Considérons l'alphabet binaire $A = \{0, 1\}^3$ de longueur $n = 3$ et le code $C = \{(0, 0, 0), (1, 1, 1)\}$. Supposons que le canal ne soit pas sans mémoire et génère le mot reçu $y = c + e \pmod{2}$ selon la loi de probabilité suivante pour le motif d'erreur $e \in \{0, 1\}^3$:

$$\Pr[e] = \begin{cases} 0,80 & \text{si } e = (0, 0, 0), \\ 0,15 & \text{si } e = (0, 1, 1), \\ 0,01 & \text{pour chacun des 6 autres motifs d'erreur d'un poids } \text{wt}(e) \in \{1, 2, 3\}. \end{cases}$$

Supposons que le mot reçu soit $y = (0, 1, 1)$.

- (a) Calculer la distance de Hamming entre y et chacun des deux mots de code de C . Quel mot de code est choisi par le décodeur au plus proche voisin ($\hat{c}_{NN}$) ?
- (b) Calculer la probabilité conditionnelle $\Pr[y | c]$ pour chacun des deux mots de code $c \in C$. Quel mot de code est choisi par le décodeur au maximum de vraisemblance ($\hat{c}_{ML}$) ?

§ 2.3 Un code à chiffre de contrôle du monde réel : le numéro de sécurité sociale français

Tous les schémas de contrôle d'erreur utilisés au quotidien ne sont pas des constructions algébriques sophistiquées ; certains des plus courants ne sont qu'une simple vérification modulaire, et il vaut la peine de les voir explicitement avant que la machinerie des chapitres suivants ne fasse paraître tout plus compliqué qu'il ne l'est réellement.

Le *numéro d'inscription au répertoire* (NIR, couramment appelé « numéro de sécurité sociale ») est un nombre N à 13 chiffres codant le sexe, l'année et le mois de naissance, un code de département (et, pour les personnes nées à l'étranger, un code pays), la commune de naissance, et un numéro d'ordre. Pour se prémunir contre les erreurs de transcription, une *clé de contrôle* $K \in \{01, \dots, 97\}$ à 2 chiffres est ajoutée, calculée comme

$$K := 97 - (N \bmod 97)$$

(avec la convention qu'un code de département « 2A » ou « 2B », pour les personnes nées en Corse, est remplacé respectivement par « 19 » ou « 18 » avant de calculer $N \bmod 97$, et que $K = 97$ plutôt que $K = 0$ lorsque $97 \mid N$). La séquence complète à 15 chiffres (N, K) est ce qui apparaît effectivement sur une carte de sécurité sociale et une fiche de paie françaises.

Exemple 2.29.

$$\underbrace{2 \ 69 \ 05 \ 34 \ 172 \ 157}_{\text{NIR}} \quad \underbrace{64}_{\text{clé de contrôle}}$$

- 2 → sexe féminin (1 pour les hommes)
- 69 → année de naissance (1969)
- 05 → mois de naissance (mai)
- 34 → département de naissance (Hérault)

- 172 → code de la commune (Montpellier)
- 157 → 157-ème personne née cette année-là et ce mois-là dans cette commune (si > 999, un peu plus compliqué)
- 64 → clé de contrôle

Algébriquement, il s'agit d'une unique *équation de contrôle de parité sur $\mathbb{Z}_{97}$* : en notant $\bar{N} := N \bmod 97$, les paires valides sont exactement celles vérifiant $\bar{N} + K \equiv 0 \pmod{97}$ — l'analogue direct, sur le module 97 au lieu de $\mathbb{F}_2$, du code de contrôle de parité binaire de la section suivante. Le module 97 n'est pas arbitraire : c'est le plus grand nombre premier inférieur à 100 (de sorte que K tienne encore sur deux chiffres), et c'est précisément la primalité qui rend les garanties de détection d'erreur du schéma (voir les exercices) démontrables plutôt qu'accidentelles.

Exercice 2.30. Vérifier la clé de contrôle avec votre propre numéro de sécurité sociale français.

Exercice 2.31 (Le NIR détecte toute erreur portant sur un seul chiffre). Supposons que le nombre véritable soit N avec la clé de contrôle $K = 97 - (N \bmod 97)$, et qu'exactly un chiffre de N soit mal transcrit (à une position $i \in \{0, \dots, 12\}$, en comptant à partir du chiffre des unités, le chiffre passant de d à $d' \neq d$), donnant un nombre corrompu $N' \neq N$, tandis que K lui-même est transmis correctement.

- Montrer que $N' - N = (d' - d) \cdot 10^i$ pour un certain entier $d' - d$ avec $1 \leq |d' - d| \leq 9$.
- En utilisant le fait que 97 est premier et ne divise pas 10 (donc $10^i \not\equiv 0 \pmod{97}$), montrer que $N' - N \not\equiv 0 \pmod{97}$.
- En déduire que recalculer la clé de contrôle à partir de N' donne $97 - (N' \bmod 97) \neq K$: une erreur portant sur un seul chiffre est *toujours* détectée.

Exercice 2.32 (Deux erreurs peuvent s'annuler). Supposons maintenant que *deux* chiffres de N soient simultanément mal transcrits, aux positions $i \neq j$, changeant respectivement de $\Delta_i, \Delta_j \in \{-9, \dots, -1, 1, \dots, 9\}$ (donc $N' - N = \Delta_i 10^i + \Delta_j 10^j$).

- Exhiber un choix explicite de i, j, Δ_i, Δ_j pour lequel $N' - N \equiv 0 \pmod{97}$ — c'est-à-dire que l'erreur passe complètement inaperçue. (Indication : fixer Δ_j et chercher parmi les 9 valeurs possibles de Δ_i celle, s'il en existe une, qui pour les i, j choisis rend la somme $\equiv 0 \pmod{97}$; ajuster i, j si la première tentative n'a pas de solution dans la plage autorisée.)
- Supposons les positions $i \neq j$ fixées et Δ_i, Δ_j indépendants et uniformes sur les 9 différences de chiffres non nulles $\{-9, \dots, -1, 1, \dots, 9\}$. Montrer que pour chacune des 9 valeurs de Δ_j , *au plus une* valeur de Δ_i parmi les 9 possibles rend l'erreur indétectable (utiliser que deux éléments distincts de $\{-9, \dots, -1, 1, \dots, 9\}$ diffèrent de moins de 97, donc ne peuvent être congrus modulo 97 que s'ils sont égaux). En déduire que la probabilité qu'une double erreur soit indétectable est au plus $9/81 = 1/9$.

§ 2.4 Premier aperçu de bons et de mauvais codes

Exemple 2.33 (Codes de répétition et de contrôle de parité). Sur $A = \mathbb{F}_q$, le *code de répétition* de longueur n , $\{(a, a, \dots, a) \mid a \in \mathbb{F}_q\}$, a pour paramètres $(n, q, n)_q$: excellente distance minimale ($d = n$), mais rendement très faible ($R = 1/n$). À l'inverse, le *code de contrôle de parité q -aire* $\{u \in \mathbb{F}_q^n \mid u_1 + \dots + u_n = 0\}$ a pour paramètres $(n, q^{n-1}, 2)_q$: excellent rendement ($R = (n-1)/n$), mais distance minimale très faible ($d = 2$) — il ne peut que *détecter* une erreur unique sans pouvoir

la corriger. La majeure partie de ce cours consiste à trouver de bons compromis entre ces deux cas extrêmes.

Exemple 2.34 (Un premier bon compromis : le code de Hamming). Pour $r \geq 2$, posons $n = 2^r - 1$. Il existe un code binaire de longueur n , à 2^{n-r} mots de code, et de distance minimale 3 : il corrige une seule erreur (mieux que aucun !) tout en ne sacrifiant que $r = \log_2(n+1)$ symboles de redondance — un taux tendant vers 1 lorsque n croît.

§ 2.5 Équivalence de codes

Renommer l'alphabet coordonnée par coordonnée, ou permuter les n positions, produit un code de paramètres identiques et de comportement de correction d'erreurs identique ; il est donc naturel d'identifier de tels codes.

Définition 2.35. Deux codes $C, C' \subseteq A^n$ sont *équivalents* si $C' = \{(\sigma_1(c_{\pi(1)}), \dots, \sigma_n(c_{\pi(n)})) \mid c \in C\}$ pour une certaine permutation $\pi \in S_n$ des coordonnées et certaines permutations $\sigma_1, \dots, \sigma_n$ de l'alphabet appliquées coordonnée par coordonnée.

Des codes équivalents ont la même taille et la même distribution des distances. La réciproque échoue de façon spectaculaire — des codes inéquivalents peuvent partager tous les mêmes paramètres (n, M, d) — un fait à garder à l'esprit chaque fois qu'un tableau (comme celui de la Section 3.5) ne liste que des paramètres et non les codes eux-mêmes.

Exercice 2.36. Démontrer que l'équivalence de codes (Définition 2.35) est une relation d'équivalence, et que des codes équivalents ont la même taille et la même distribution des distances $(A_0, A_1, \dots, A_n)$, où $A_i := |\{(x, y) \in C^2 : d(x, y) = i\}|/|C|$. Trouver un exemple de deux codes *inéquivalents* partageant les mêmes paramètres (n, M, d) .

Codes linéaires

Désormais, nous nous spécialisons aux alphabets qui sont des corps finis, $A = \mathbb{F}_q$, et à C un $\mathbb{F}_q$ -sous-espace de $\mathbb{F}_q^n$: c'est le cadre dans lequel deux astuces calculatoires portent leurs fruits : une base (plutôt qu'une liste complète) décrit le code, et une application linéaire (plutôt qu'une table de correspondance) décrit l'encodeur.

§ 3.1 Matrices génératrice et de contrôle de parité, dualité

Definition 3.1. Un *code linéaire* C de longueur n et de dimension k sur $\mathbb{F}_q$ est un $\mathbb{F}_q$ -sous-espace de $\mathbb{F}_q^n$ de dimension k ; on dit que C est un code $[n, k]_q$, ou un code $[n, k, d]_q$ si de plus sa distance minimale est d .

Proposition 3.2. Si C est un code linéaire de dimension k sur $\mathbb{F}_q$, alors il possède q^k éléments.

Démonstration. Une base du code est donnée par k vecteurs. Toute combinaison linéaire de ces vecteurs peut être obtenue en sommant ces vecteurs multipliés chacun par un élément de $\mathbb{F}_q$. Il y a donc au total $|\mathbb{F}_q|^k = q^k$ choix possibles. ■

Proposition 3.3. Si C est un code linéaire, alors

$$d(C) = \min_{0 \neq c \in C} \text{wt}(c).$$

Démonstration. Puisque $d(x, y) = \text{wt}(x - y)$, le résultat découle immédiatement du fait que C est un sous-groupe de $(\mathbb{F}_q^n, +)$. ■

La proposition précédente implique que le calcul de la distance minimale pour un code linéaire ramène les $\binom{|C|}{2}$ calculs de distances deux à deux nécessaires pour un code générique à « seulement » $|C| - 1$ calculs de poids.

Remark 3.4 (Combien de bits un code coûte-t-il réellement à stocker ?). Il vaut la peine de rendre complètement quantitative l'économie « base plutôt que liste complète » du paragraphe d'ouverture du chapitre. Un code générique $C \subseteq \mathbb{F}_q^n$ de taille M n'a, dans le pire des cas, aucun raccourci du tout : écrire tous les mots de code coûte $Mn \log_2 q$ bits. Un code linéaire $[n, k]_q$, en revanche, possède $M = q^k$ mots de code mais est entièrement déterminé par une matrice génératrice $k \times n$, ne coûtant que

$$kn \log_2 q \text{ bits} \quad \text{au lieu de} \quad q^k n \log_2 q \text{ bits.}$$

Pour $k = 100$, $n = 200$, $q = 2$: la matrice génératrice coûte $100 \times 200 = 20\,000$ bits (environ 2,5 Ko) ; la liste naïve nécessiterait $2^{100} \times 200$ bits — bien plus que le nombre d'atomes dans l'univers observable, sans même

parler de ce qui serait stockable. Cet écart exponentiel contre polynomial, et non une simple préférence esthétique pour l'algèbre, est la véritable raison pour laquelle pratiquement tous les codes utilisés en pratique sont linéaires ou très proches de l'être : un code non linéaire $A_q(n, d)$ -optimal (Définition 2.10) peut occasionnellement battre le meilleur code linéaire d'un petit facteur constant en *taille* (Section 3.5), mais il n'offre en général aucune description aussi compacte, ni aucun algorithme de décodage efficace.

Exercice 3.5. Pour chaque code ci-dessous, déterminer s'il est linéaire (c'est-à-dire un $\mathbb{F}_q$ -sous-espace de $\mathbb{F}_q^n$) ou non, en justifiant votre réponse soit en exhibant une base (si linéaire), soit en exhibant un échec concret de la clôture, tel que deux mots de code dont la somme n'appartient pas au code (sinon). Rappelons que le fait que $|C|$ soit une puissance de q est *nécessaire* pour la linéarité mais, comme le montrent certains des exemples ci-dessous, loin d'être *suffisant*.

- (a) Les deux codes *triviaux* $C = \{0\} \subseteq \mathbb{F}_q^n$ et $C = \mathbb{F}_q^n$, pour q, n arbitraires.
- (b) $C = \{000, 100, 010, 111\} \subseteq \mathbb{F}_2^3$.
- (c) $C = \{000, 110, 101, 011\} \subseteq \mathbb{F}_2^3$.
- (d) $C = \{(0, 0), (1, 1), (2, 0), (0, 2), (1, 0)\} \subseteq \mathbb{F}_3^2$.
- (e) $C = \{(a, b, a + 2b, 3a - b) \mid a, b \in \mathbb{F}_5\} \subseteq \mathbb{F}_5^4$.
- (f) $C = \{(a, b, ab) \mid a, b \in \mathbb{F}_3\} \subseteq \mathbb{F}_3^3$.
- (g) $C = \{(a, b, a^2 + b) \mid a, b \in \mathbb{F}_3\} \subseteq \mathbb{F}_3^3$.
- (h) $C = \{(a, b, a^2 + b) \mid a, b \in \mathbb{F}_2\} \subseteq \mathbb{F}_2^3$.

Définition 3.6 (Matrices génératrice et de contrôle de parité). Une *matrice génératrice* $G \in \mathbb{F}_q^{k \times n}$ de C est toute matrice dont les lignes forment une base de C , de sorte que $C = \{uG : u \in \mathbb{F}_q^k\}$. Une *matrice de contrôle de parité* $H \in \mathbb{F}_q^{(n-k) \times n}$ de C est une matrice de rang plein telle que $C = \{c \in \mathbb{F}_q^n : Hc^T = 0\}$.

Exercice 3.7. La matrice

$$G = \begin{pmatrix} 1 & 2 & 2 & 4 \\ 3 & 2 & 1 & 0 \\ 2 & 1 & 4 & 4 \end{pmatrix}$$

est-elle une matrice génératrice d'un code linéaire sur $\mathbb{F}_5$? Et sur $\mathbb{F}_7$?

Pour un code linéaire avec matrice génératrice $G \in \mathbb{F}_q^{k \times n}$, l'**encodage** d'un message $u \in \mathbb{F}_q^k$ est obtenu comme le produit $uG \in \mathbb{F}_q^n$.

Définition 3.8 (Forme systématique). Une matrice génératrice G est sous *forme systématique* si $G = [I_k \mid A]$ pour un certain $A \in \mathbb{F}_q^{k \times (n-k)}$, c'est-à-dire que les k premières colonnes forment la matrice identité. Dans ce cas, encoder un message $u \in \mathbb{F}_q^k$ en $c = uG = (u \mid uA)$ reproduit littéralement u inchangé dans les k premières coordonnées (les *symboles d'information*) et ajoute $n - k$ *symboles de parité* redondants uA .

Exemple 3.9. Le code

$C = \{(0, 0, 0), (1, 2, 2), (2, 1, 1), (0, 1, 2), (0, 2, 1), (1, 0, 1), (2, 0, 2), (1, 2, 0), (2, 1, 0)\} \subseteq \mathbb{F}_3^3$ est linéaire de dimension 2. La matrice

$$G = \begin{pmatrix} 1 & 2 & 2 \\ 2 & 0 & 2 \end{pmatrix}$$

est une matrice génératrice de C et

$$G = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

en est la matrice génératrice sous forme systématique.

Proposition 3.10. *Tout code $[n, k]_q$ C possède une matrice génératrice sous forme systématique, éventuellement après permutation des n coordonnées (c'est-à-dire que C est toujours équivalent, via une permutation de coordonnées, à un code possédant une matrice génératrice systématique).*

Démonstration. Partons d'une matrice génératrice quelconque G_0 . Puisque G_0 est de rang k , certaines k de ses colonnes sont linéairement indépendantes; permutons les coordonnées pour que ce soient les k premières colonnes, formant une sous-matrice inversible M de taille $k \times k$. En multipliant à gauche par M^{-1} , on obtient $G := M^{-1}G_0 = [I_k \mid A]$, qui est toujours une matrice génératrice du code (à coordonnées permutes), puisque la multiplication à gauche par une matrice inversible ne fait que changer le choix de la base. ■

Proposition 3.11. *Si $G = [I_k \mid A]$ est une matrice génératrice systématique de C , alors*

$$H := [-A^T \mid I_{n-k}]$$

est une matrice de contrôle de parité de C . Réciproquement, si $H = [B \mid I_{n-k}]$ est une matrice de contrôle de parité systématique quelconque, alors $G := [I_k \mid -B^T]$ est une matrice génératrice systématique de C .

Démonstration. H est de rang $n - k$ (elle contient I_{n-k}), donc elle est bien une matrice de contrôle de parité dès lors que l'on vérifie $GH^T = 0$:

$$GH^T = [I_k \mid A] \begin{pmatrix} -A \\ I_{n-k} \end{pmatrix} = -A + A = 0.$$

Par la définition du code dual, H est alors automatiquement une matrice génératrice de $C^\perp$, et elle est visiblement sous forme systématique (bloc identité à droite). La réciproque est le même calcul lu à l'envers, en utilisant $(C^\perp)^\perp = C$. ■

Exemple 3.12. Pour $C \subseteq \mathbb{F}_5^5$ de matrice génératrice systématique $G = \begin{pmatrix} 1 & 0 & 0 & 1 & 2 \\ 0 & 1 & 0 & 2 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix} = [I_3 \mid A]$, la Proposition 3.11 donne immédiatement $H = [-A^T \mid I_2] = \begin{pmatrix} -1 & -2 & -1 & 1 & 0 \\ -2 & -1 & -1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 3 & 4 & 1 & 0 \\ 3 & 4 & 4 & 0 & 1 \end{pmatrix}$.

Exercice 3.13. Donner la matrice génératrice sous forme systématique du code $C = \langle (3, 4, 0, 2), (6, 6, 1, 5) \rangle \subseteq \mathbb{F}_7^4$ ainsi qu'une matrice de contrôle de parité.

Définition 3.14. Le code dual de C est $C^\perp := \{x \in \mathbb{F}_q^n : \langle x, c \rangle = 0 \forall c \in C\}$, où $\langle x, y \rangle = \sum x_i y_i$. Si G, H sont des matrices génératrice et de contrôle de parité pour C , alors H, G sont des matrices de contrôle de parité et génératrice pour $C^\perp$, et $(C^\perp)^\perp = C$. En particulier $\dim C^\perp = n - \dim C$.

Puisque $C^\perp$ est le sous-espace linéaire orthogonal à C , on a

$$\dim C^\perp = n - \dim C.$$

Remark 3.15. Lorsqu'on travaille avec la métrique euclidienne usuelle sur les réels, on sait que l'intersection de sous-espaces orthogonaux est triviale, c'est-à-dire qu'elle ne contient que le vecteur nul. Ici au contraire, il est possible d'avoir des codes qui s'intersectent de façon non triviale avec leur dual. Une telle intersection est appelée le *hull* d'un code linéaire. L'exercice suivant demande de les calculer.

Exercice 3.16. Hull Soit $n = 4$. Pour chaque $0 \leq k \leq n$, construire un code C de longueur n tel que $\dim C \cup C^\perp = k$, si possible. Faire de même pour $n = 5$.

Proposition 3.17 (Poids et matrice de contrôle de parité). *C a une distance minimale $\geq d$ si et seulement si toute famille de $d - 1$ colonnes d'une matrice de contrôle de parité H est linéairement indépendante.*

Démonstration. Un mot de code non nul c de poids w vérifie $Hc^T = 0$, c'est-à-dire qu'une combinaison $\mathbb{Z}_q$ -linéaire, avec pour coefficients les entrées non nulles de c , des w colonnes de H indexées par $\text{Supp}(c)$ s'annule : les mots de code de poids w correspondent donc exactement aux familles *dépendantes* de w colonnes de H . ■

Exercice 3.18. Soit $C \subseteq \mathbb{F}_3^5$ le code linéaire de matrice génératrice

$$G = \begin{pmatrix} 1 & 0 & 0 & 1 & 2 \\ 0 & 1 & 0 & 2 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

- (a) Écrire une matrice de contrôle de parité H pour C .
- (b) À l'aide de H , déterminer $d(C)$ (Proposition 3.17).
- (c) Donner les paramètres $[n, k, d]_3$ de C et de son dual $C^\perp$.

§ 3.2 Somme et intersection de codes

Deux opérations élémentaires combinent une *paire* existante de codes de même longueur pour en former un nouveau. Elles interviennent constamment en coulisses : la concaténation parallèle au cœur des turbocodes (Chapitre 11) est construite à partir de deux codes partageant les mêmes bits d'information, et les identités de dualité ci-dessous sont précisément l'outil qui permet de passer entre les descriptions par matrice génératrice et par matrice de contrôle de parité de tels codes combinés.

Définition 3.19. Soient $C_1, C_2 \subseteq \mathbb{F}_q^n$ deux codes de même longueur n . Leur *somme* est

$$C_1 + C_2 := \{c_1 + c_2 : c_1 \in C_1, c_2 \in C_2\} \subseteq \mathbb{F}_q^n,$$

et leur *intersection* $C_1 \cap C_2$ est l'intersection ensembliste ordinaire. Les deux sont encore des codes linéaires de longueur n (des sous-espaces de $\mathbb{F}_q^n$).

Proposition 3.20 (Dimension). $\dim(C_1 + C_2) + \dim(C_1 \cap C_2) = \dim(C_1) + \dim(C_2)$.

Démonstration. C'est la formule des dimensions de Grassmann pour les sous-espaces d'un espace vectoriel, appliquée à $C_1, C_2 \leq \mathbb{F}_q^n$: on étend une base de $C_1 \cap C_2$ en une base de C_1 , et séparément en une base

de C_2 ; les deux extensions, ensemble avec la base de l'intersection, forment une base de $C_1 + C_2$ (une brève vérification de génération et d'indépendance linéaire, exactement comme en algèbre linéaire ordinaire), ce qui donne $\dim(C_1 + C_2) = \dim(C_1) + \dim(C_2) - \dim(C_1 \cap C_2)$. ■

Une matrice génératrice de $C_1 + C_2$ s'obtient en empilant les matrices génératrices de C_1 et C_2 et en éliminant les lignes dépendantes ; une matrice génératrice de $C_1 \cap C_2$ s'obtient, de façon duale, en empilant les matrices de contrôle de parité de C_1 et C_2 et en lisant une base du noyau conjoint. Cette dualité entre les deux constructions est exacte :

Proposition 3.21 (Dualité). $(C_1 + C_2)^\perp = C_1^\perp \cap C_2^\perp$ et $(C_1 \cap C_2)^\perp = C_1^\perp + C_2^\perp$.

Démonstration. Pour la première identité : $x \perp (C_1 + C_2)$ si et seulement si $\langle x, c_1 + c_2 \rangle = 0$ pour tous $c_1 \in C_1, c_2 \in C_2$; en prenant $c_2 = 0$ (resp. $c_1 = 0$), on voit que cela force $x \perp C_1$ et $x \perp C_2$ séparément, et réciproquement $x \perp C_1, C_2$ donne clairement $x \perp (C_1 + C_2)$ par linéarité ; donc $(C_1 + C_2)^\perp = C_1^\perp \cap C_2^\perp$. La seconde identité s'obtient en appliquant la première à $C_1^\perp, C_2^\perp$ à la place de C_1, C_2 et en utilisant $(C^\perp)^\perp = C$. ■

Proposition 3.22 (Distance). $d(C_1 \cap C_2) \geq \max(d(C_1), d(C_2))$ dès que $C_1 \cap C_2 \neq \{0\}$, et $d(C_1 + C_2) \leq \min(d(C_1), d(C_2))$ dès que $C_1 + C_2$ possède au moins deux mots de code.

Démonstration. Tout mot de code non nul de $C_1 \cap C_2$ est, en particulier, un mot de code non nul de C_1 et de C_2 séparément, donc de poids $\geq d(C_1)$ et $\geq d(C_2)$, ce qui donne la première inégalité. Pour la seconde, $C_1 \subseteq C_1 + C_2$ et $C_2 \subseteq C_1 + C_2$, donc l'Exercice 2.9(a) (l'inclusion contrôle la distance) s'applique deux fois : $d(C_1 + C_2) \leq d(C_1)$ et $d(C_1 + C_2) \leq d(C_2)$. ■

Exemple 3.23. Soient $q = 2, n = 4, C_1 := \langle (1, 1, 0, 0), (0, 0, 1, 1) \rangle$ et $C_2 := \langle (1, 0, 1, 0), (0, 1, 0, 1) \rangle$, aucun n'étant contenu dans l'autre (par exemple $(1, 1, 0, 0) \in C_1 \setminus C_2$). En listant les mots de code, $C_1 = \{0000, 1100, 0011, 1111\}$ et $C_2 = \{0000, 1010, 0101, 1111\}$, donc $C_1 \cap C_2 = \{0000, 1111\}$, le code de répétition $[4, 1, 4]_2$, et l'on calcule que $C_1 + C_2$ est le code de contrôle de parité $[4, 3, 2]_2$. La formule de Grassmann se vérifie : $\dim(C_1 + C_2) + \dim(C_1 \cap C_2) = 3 + 1 = 4 = 2 + 2 = \dim(C_1) + \dim(C_2)$. Conformément à la Proposition 3.22, $d(C_1 \cap C_2) = 4 \geq \max(2, 2)$ (strictement plus grand ici) et $d(C_1 + C_2) = 2 \leq \min(2, 2)$ (égalité ici).

Exercice 3.24. Soient $q = 3, n = 4, C_1 := \langle (1, 1, 0, 0), (0, 0, 1, 1) \rangle$ et $C_2 := \langle (1, 1, 0, 0), (0, 1, 1, 2) \rangle$ deux codes linéaires sur $\mathbb{F}_3$.

- Lister les $3^2 = 9$ mots de code de C_1 et de C_2 , puis déterminer $C_1 \cap C_2$ directement à partir de ces deux listes.
- Donner une matrice génératrice sous forme échelonnée réduite pour $C_1 + C_2$, et vérifier numériquement la formule $\dim(C_1 + C_2) = \dim C_1 + \dim C_2 - \dim(C_1 \cap C_2)$.
- Calculer $d(C_1), d(C_2), d(C_1 \cap C_2)$ et $d(C_1 + C_2)$, puis vérifier les inégalités sur la distance minimale de la somme et de l'intersection.

§ 3.3 Poinçonnage et raccourcissement : fabriquer de nouveaux codes

Deux autres opérations élémentaires permettent d'échanger de la longueur contre de la dimension ou de la distance sans repartir d'une construction entièrement nouvelle ; elles sont utilisées constamment, à la fois pour construire des codes pratiques de longueur « irrégulière » à partir d'un beau code algébrique, et à l'intérieur de plusieurs des preuves de bornes du chapitre suivant.

Définition 3.25 (Poinçonnage). Fixons une position de coordonnée $i \in \{1, \dots, n\}$. Le code poinçonné C^{*i} est obtenu en supprimant la i -ème coordonnée de chaque mot de code de C :

$$C^{*i} := \{(c_1, \dots, c_{i-1}, c_{i+1}, \dots, c_n) : c \in C\} \subseteq \mathbb{F}_q^{n-1}.$$

Proposition 3.26. Soit C un code $[n, k, d]_q$ avec $d \geq 2$. Alors C^{*i} est un code $[n-1, k, d']_q$ avec $d' \in \{d-1, d\}$; plus précisément, $d' = d-1$ si C contient un mot de code de poids minimal non nul en position i , et $d' = d$ sinon. (Poinçonner en plusieurs positions à la fois se définit de la même manière, en les supprimant toutes simultanément.)

Démonstration. La suppression est une surjection linéaire $C \rightarrow C^{*i}$; elle est injective sauf si deux mots de code de C coïncident partout sauf éventuellement en position i , ce qui, puisque $d \geq 2$, ne peut se produire pour deux mots de code *distincts* ne différant qu'à cette position sans différer aussi ailleurs — une brève vérification montre que le noyau est trivial lorsque $d \geq 2$, donc $\dim C^{*i} = k$. Chaque mot de code perd au plus une coordonnée non nulle lors de la suppression, d'où $d' \geq d-1$; et $d' \geq d$ n'est impossible que si un mot de code de poids minimal est non nul exactement en i , auquel cas $d' = d-1$ est atteint par ce mot et ne peut pas descendre davantage d'après la phrase précédente. ■

Définition 3.27 (Raccourcissement). Fixons une position de coordonnée i . Le code raccourci C_i est

$$C_i := \{(c_1, \dots, c_{i-1}, c_{i+1}, \dots, c_n) : c \in C, c_i = 0\} \subseteq \mathbb{F}_q^{n-1}.$$

Proposition 3.28. Soit C un code $[n, k, d]_q$. Alors C_i est un code $[n-1, k', d']_q$ avec $k' \in \{k-1, k\}$ (typiquement $k-1$, et exactement k seulement dans le cas dégénéré où tout mot de code s'annule déjà en position i) et $d' \geq d$.

Proposition 3.29 (Poinçonnage et raccourcissement sont duaux l'un de l'autre). Pour tout i , $(C^{*i})^\perp = (C^\perp)_i$ et $(C_i)^\perp = (C^\perp)^{*i}$.

Démonstration. Un vecteur $x \in \mathbb{F}_q^{n-1}$ (vu avec un 0 réinséré en position i) est orthogonal à tout mot de C^{*i} exactement lorsque son extension par zéro est orthogonale à tout mot de C ayant pu le produire par suppression, c'est-à-dire à tout mot de C — mais tout $y \in C^\perp$ qui est nul en i réalise déjà cela, ce qui donne $(C^\perp)_i \subseteq (C^{*i})^\perp$, et un décompte de dimensions (utilisant les Propositions 3.26–3.28) montre l'égalité. ■

Exemple 3.30. Poinçonner le code de répétition $[n, 1, n]_2$ en une position donne le code de répétition $[n-1, 1, n-1]_2$ (le taux augmente très légèrement, la distance chute exactement de 1). Raccourcir le code de contrôle de parité $[n, n-1, 2]_2$ en une position donne le code de contrôle de parité $[n-1, n-2, 2]_2$: cohérent avec la Proposition 3.29, puisque parité et répétition sont duaux l'un de l'autre.

Exercice 3.31. Soit C le code $[6, 3]_2$ de matrice génératrice $G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$.

- Poinçonner C en position 5 : donner une matrice génératrice et les paramètres de C^{*5} .
- Raccourcir C en position 5 : donner une matrice génératrice et les paramètres de C_5 .
- Vérifier directement, sur cet exemple, la relation de dualité $(C^{*5})^\perp = (C^\perp)_5$ de la Proposition 3.29.

La construction suivante peut être vue comme l'inverse du raccourcissement/poinçonnage, puisqu'elle insère une coordonnée supplémentaire et redondante.

Definition 3.32 (Extension). Étant donné C un code $[n, k, d]_q$, son *code étendu* est $\widehat{C} := \{(c, c_{n+1}) : c \in C, \sum_i c_i + c_{n+1} = 0\}$, un code $[n+1, k, d^+]_q$ avec $d^+ \in \{d, d+1\}$.

Il existe bien d'autres constructions qui, à partir d'un ou plusieurs codes linéaires, en fournissent un nouveau ; nous mentionnons simplement le produit tensoriel des codes et la somme de Plotkin.

§ 3.4 Une galerie de familles classiques

Example 3.33. En fait, les codes de répétition et de parité sont linéaires !

Example 3.34 (Codes de Hamming et simplexe). Pour $r \geq 2$, $n = 2^r - 1$, soit $H_r \in \mathbb{F}_2^{r \times n}$ ayant pour colonnes tous les vecteurs non nuls de $\mathbb{F}_2^r$, chacun exactement une fois. Le code de matrice de contrôle de parité H_r est le *code de Hamming* $\mathcal{H}_r$, un code $[n, n-r, 3]_2$: aucune colonne de H_r n'est nulle et deux colonnes ne sont jamais égales, donc par la Proposition 3.17, $d \geq 3$; et trois colonnes quelconques dont la somme s'annule (qui doivent exister puisque $r < n$) montrent que $d = 3$. Son dual, le *code simplexe* $\mathcal{S}_r = \mathcal{H}_r^\perp$, est un code à poids constant : tout mot de code non nul a un poids exactement égal à 2^{r-1} , ce que l'on déduit directement du fait qu'une forme linéaire non nulle sur $\mathbb{F}_2^r$ s'annule sur exactement la moitié des vecteurs non nuls.

Exercice 3.35. Montrer que raccourcir le code de Hamming binaire $\mathcal{H}_r$ en une seule coordonnée donne toujours un code linéaire qui corrige encore une erreur, mais qui n'est en général plus lui-même un code de Hamming (sa longueur n'est plus de la forme $2^{r'} - 1$). Calculer les paramètres exacts $[n', k', d']_2$ de ce code raccourci en fonction de r .

Exercice 3.36. Démontrer que tout mot de code non nul du code simplexe $\mathcal{S}_r$ a un poids exactement égal à 2^{r-1} (compléter l'argument en une ligne esquissé dans le texte : une forme linéaire non nulle sur $\mathbb{F}_2^r$ s'annule sur exactement la moitié des $2^r - 1$ vecteurs non nuls). En déduire l'énumérateur des poids complet de $\mathcal{S}_r$.

Example 3.37 (Codes de Reed–Muller). En identifiant les points de $\mathbb{F}_2^m$ aux 2^m affectations de vérité possibles, le code $\text{RM}(r, m)$ d'ordre r est constitué des vecteurs d'évaluation de tous les polynômes booléens de degré $\leq r$ sur $\mathbb{F}_2^m$. C'est un code $[2^m, \sum_{i=0}^r \binom{m}{i}, 2^{m-r}]_2$, et $\text{RM}(r, m)^\perp = \text{RM}(m-r-1, m)$. Le code d'ordre 1, $\text{RM}(1, m)$, a été utilisé, dans une instance de longueur 32, sur la mission Mariner 9 de la NASA — un exemple précoce et concret de la théorie des codes quittant le tableau noir.

Exercice 3.38. Montrer que $\text{RM}(1, m)^\perp = \text{RM}(m-2, m)$ est cohérent avec la formule de dualité générale $\text{RM}(r, m)^\perp = \text{RM}(m-r-1, m)$, et vérifier le décompte des dimensions $\dim \text{RM}(r, m) + \dim \text{RM}(m-r-1, m) = 2^m$ directement à partir de la formule $\dim \text{RM}(r, m) = \sum_{i=0}^r \binom{m}{i}$, en utilisant l'identité binomiale $\sum_{i=0}^m \binom{m}{i} = 2^m$.

Exemple 3.39 (Les codes de Golay). Il existe un code binaire $[24, 12, 8]_2$ essentiellement unique, auto-dual, appelé le *code de Golay binaire étendu* $\mathcal{G}_{24}$; le poinçonner une fois donne le *code de Golay* $\mathcal{G}_{23}$, un code $[23, 12, 7]_2$ qui est *parfait* (égalité dans la borne de Hamming du chapitre suivant). Un analogue ternaire, $\mathcal{G}_{12}$, est un code parfait $[11, 6, 5]_3$. Les deux ont été découverts par Golay en 1949. Ils ont des applications bien au-delà de la théorie des codes : en théorie des groupes et dans les problèmes de réseaux/d'empilement de sphères (le réseau de Leech est construit à partir de $\mathcal{G}_{24}$).

D'autres familles des codes, comme par exemple les codes de Reed–Solomon, méritent plus d'espace et seront abordés plus avant.

Exercice 3.40. Dans le prochain chapitre nous allons justifier qu'aucun code linéaire binaire $[5, 3, 3]_2$ n'existe pas. Construire un code optimal $[5, 3, 2]_2$ en appliquant un raccourcissement suivi d'un poinçonnage à un code binaire classique remarquable.

§ 3.5 Les codes non linéaires peuvent tout de même l'emporter

Le Chapitre 2 a introduit $A_q(n, d)$ — la plus grande taille possible pour *n'importe quel* code, linéaire ou non, de longueur donnée et de distance minimale garantie (Définition 2.10) — comme la quantité combinatoire centrale de tout le sujet. Maintenant que les codes linéaires et leurs paramètres $[n, k, d]_q$ sont disponibles pour comparaison, il vaut la peine de le dire clairement : pour certains paramètres, aucun code linéaire n'est optimal.

Exemple 3.41 (Le code de Best). $A_2(8, 3) = 20$, réalisé par un code binaire non linéaire découvert par M. Best (aussi appelé le code de Julin). En revanche, tout code *linéaire* $[8, k, 3]_2$ vérifie $k \leq 4$ (une conséquence des certaines bornes supérieurs), c'est-à-dire au plus 16 mots de code — strictement moins que 20. Donc pour ces paramètres, aucun code linéaire n'est optimal, même si 16 est déjà honorablement proche de 20.

Exemple 3.42 (Le code de Nordstrom–Robinson). Il existe un code binaire non linéaire de longueur 16, avec 256 mots de code et de distance minimale 6, découvert indépendamment par Nordstrom et Robinson (1967). Aucun code linéaire $[16, k, 6]_2$ n'a $2^k \geq 256 = 2^8$ mots de code : le meilleur code *linéaire* avec ces paramètres a pour dimension $k = 7$, soit seulement 128 mots de code — le code non linéaire est exactement *deux fois* plus grand.

Remark 3.43 (Où trouver le record actuel). Les deux exemples ci-dessus sont des phénomènes exceptionnels à petits paramètres plutôt qu'une règle générale : asymptotiquement (taux fixé, $n \rightarrow \infty$), les meilleures constructions connues munies d'encodeurs et de décodeurs explicites et efficaces sont linéaires ou proches de l'être. Les codes non linéaires importent surtout pour les petits paramètres tabulés, où chaque mot de code supplémentaire compte. En conséquence, maintenant que les codes linéaires sont sur la table, il vaut la peine de savoir où consulter les meilleurs paramètres *actuellement connus* plutôt que de se fier à une liste figée dans un manuel : les tables de Markus Grassl sur <http://www.codetables.de> constituent la référence standard, activement maintenue, pour les meilleurs codes linéaires connus $[n, k, d]_q$ pour tout $q \leq 9$, incluant des constructions explicites, et sont continuellement mises à jour à mesure que de nouveaux codes sont découverts.

Corps finis

Chaque code linéaire de ce cours vit dans $\mathbb{F}_q^n$, et plusieurs arguments proposés se sont appuyés sur des propriétés arithmétiques spécifiques de $\mathbb{F}_q$. Cet annexe rassemble, en gardant un œil sur ce qui est réellement utilisé dans le texte principal plutôt que sur la généralité, les faits sur les corps finis dont un lecteur de ce cours a besoin, et rien de plus. Pour la théorie complète, la référence classique est l'ouvrage de Lidl et Niederreiter, *Finite Fields*.

§ A.1 Pourquoi ne pas simplement utiliser $\mathbb{Z}/n\mathbb{Z}$?

Tout dans ce cours nécessite un alphabet sur lequel on peut additionner *et* multiplier, et où chaque élément non nul possède un inverse multiplicatif — cette dernière propriété est utilisée, par exemple, chaque fois que nous normalisons un coefficient dominant, inversons une matrice de Vandermonde, etc. L'anneau $\mathbb{Z}/n\mathbb{Z}$ possède cette propriété exactement lorsque n est premier : pour n composé, $\mathbb{Z}/n\mathbb{Z}$ possède des diviseurs de zéro ($2 \cdot 3 = 0$ dans $\mathbb{Z}/6\mathbb{Z}$) et certains éléments non nuls ne sont pas inversibles. Les modules premiers $\mathbb{F}_p := \mathbb{Z}/p\mathbb{Z}$ sont donc nos premiers exemples de corps finis — mais ils ne nous fournissent que des alphabets de taille première, et les codes cycliques du Chapitre 6, en particulier, ont souvent besoin d'un corps de taille $q = 2^r$ ou $q = 3^r$ pour contenir suffisamment de racines de l'unité. Nous avons besoin de corps de tout ordre puissance d'un nombre premier $q = p^e$, et pas seulement d'ordre premier.

§ A.2 Construction de $\mathbb{F}_4$ à la main

Avant d'énoncer quoi que ce soit en général, fabriquons un corps à $4 = 2^2$ éléments, puisque $\mathbb{Z}/4\mathbb{Z}$ lui-même ne convient pas (ce n'est même pas un corps : $2 \cdot 2 = 0$).

Considérons l'anneau des polynômes $\mathbb{F}_2[X]$, et à l'intérieur le polynôme $f(X) = X^2 + X + 1$. On vérifie directement que f n'a pas de racine dans $\mathbb{F}_2$ ($f(0) = 1$, $f(1) = 1$), donc — étant de degré 2 — f est *irréductible* sur $\mathbb{F}_2$: il ne peut pas être factorisé en deux facteurs linéaires, et ne peut donc pas être factorisé du tout. Formons l'anneau quotient

$$\mathbb{F}_4 := \mathbb{F}_2[X]/(f(X)),$$

dont les éléments sont les (classes de) polynômes de degré < 2 à coefficients dans $\mathbb{F}_2$: en notant α la classe de X , chaque élément de $\mathbb{F}_4$ s'écrit de manière unique $a + b\alpha$ avec $a, b \in \{0, 1\}$, et α vérifie $\alpha^2 + \alpha + 1 = 0$, c'est-à-dire $\alpha^2 = \alpha + 1$ (l'arithmétique est modulo 2, donc $-1 = 1$). Cela nous donne les quatre éléments $0, 1, \alpha, \alpha + 1$, avec une addition composante par composante mod 2 et une multiplication réduite en utilisant $\alpha^2 = \alpha + 1$:

$$\alpha \cdot (\alpha + 1) = \alpha^2 + \alpha = (\alpha + 1) + \alpha = 1,$$

ainsi α et $\alpha + 1$ sont inverses l'un de l'autre — chaque élément non nul de $\mathbb{F}_4$ possède bien un inverse, et on vérifie les autres axiomes de corps tout aussi naturellement. Remarquons également que $\alpha^3 = \alpha \cdot \alpha^2 =$

$\alpha(\alpha + 1) = 1$: les trois éléments non nuls de $\mathbb{F}_4$ sont exactement $\alpha^0, \alpha^1, \alpha^2$, c'est-à-dire que α engendre le groupe multiplicatif $\mathbb{F}_4^*$. Ce n'est pas une coïncidence, comme l'expliquent les deux sections suivantes.

§ A.3 La construction générale

Theorem A.1. *Tout corps fini F vérifie $|F| = p^e$ pour un certain nombre premier p (sa caractéristique) et un certain entier $e \geq 1$.*

Démonstration. L'homomorphisme d'anneaux $\mathbb{Z} \rightarrow F, m \mapsto m \cdot 1_F$, ne peut pas être injectif (F est fini), donc son noyau est un idéal non nul $p\mathbb{Z}$ de $\mathbb{Z}$; comme F n'a pas de diviseur de zéro, p doit être premier, et $\mathbb{Z}/p\mathbb{Z} \hookrightarrow F$ réalise F comme un espace vectoriel sur le sous-corps $\mathbb{F}_p := \mathbb{Z}/p\mathbb{Z}$ — nécessairement de dimension finie e , d'où $|F| = p^e$. ■

Theorem A.2 (Existence et unicité). *Pour toute puissance d'un nombre premier $q = p^e$, il existe un corps d'ordre q , et deux corps d'ordre q quelconques sont isomorphes. On le note $\mathbb{F}_q$.*

Démonstration. *Existence.* On montre (un fait classique, bien que légèrement élaboré, de la théorie — voir la référence ci-dessus) que $\mathbb{F}_p[X]$ contient toujours un polynôme irréductible unitaire f de tout degré prescrit e ; puis, exactement comme nous l'avons fait pour $\mathbb{F}_4$ ci-dessus, $\mathbb{F}_p[X]/(f)$ est un corps (un quotient d'un anneau principal par un idéal maximal, puisque les éléments irréductibles engendrent des idéaux maximaux) d'ordre p^e , ses éléments étant les classes de polynômes de degré $< e$. *Unicité.* Tout corps d'ordre p^e peut être montré comme étant (isomorphe au) corps de décomposition de $X^{p^e} - X$ sur $\mathbb{F}_p$ — ses p^e éléments sont précisément les p^e racines de ce seul polynôme — et les corps de décomposition d'un polynôme donné sur un corps donné sont uniques à un isomorphisme près, un fait fondamental de la théorie de Galois. ■

Remark A.3 (La construction, sur le plan opérationnel). Pour calculer dans $\mathbb{F}_{p^e}$ à la main : (1) trouver un polynôme irréductible unitaire $f \in \mathbb{F}_p[X]$ de degré e ; (2) soit α une racine de f (formellement, la classe de X dans $\mathbb{F}_p[X]/(f)$); (3) chaque élément s'écrit de façon unique $a_0 + a_1\alpha + \dots + a_{e-1}\alpha^{e-1}$ avec $a_i \in \mathbb{F}_p$; (4) réduire toute puissance α^j , $j \geq e$, à l'aide de la relation $f(\alpha) = 0$ résolue pour le terme de plus haut degré. C'est exactement ce que nous avons fait pour $q = 4$ ci-dessus.

Exemple A.4. Construisons $\mathbb{F}_9 = \mathbb{F}_{3^2}$. Le polynôme $f(X) = X^2 + 1 \in \mathbb{F}_3[X]$ n'a pas de racine dans $\mathbb{F}_3$ ($0^2 + 1 = 1, 1^2 + 1 = 2, 2^2 + 1 = 2$), il est donc irréductible. Avec $\alpha^2 = -1 = 2$, les neuf éléments de $\mathbb{F}_9$ sont $a + b\alpha$, $a, b \in \{0, 1, 2\}$. Par exemple, $(1 + \alpha)(2 + \alpha) = 2 + \alpha + 2\alpha + \alpha^2 = 2 + 3\alpha + 2 = 4 + 3\alpha = 1$ (en utilisant $3 = 0$ et $\alpha^2 = 2$) : ainsi $1 + \alpha$ et $2 + \alpha$ sont inverses l'un de l'autre.

§ A.4 Le groupe multiplicatif et les éléments primitifs

Proposition A.5. *Pour toute puissance d'un nombre premier q , le groupe multiplicatif $\mathbb{F}_q^* = \mathbb{F}_q \setminus \{0\}$ est cyclique (d'ordre $q - 1$).*

Un élément α engendrant $\mathbb{F}_q^*$ (équivalamment : vérifiant $\alpha^{q-1} = 1$ et aucune puissance stricte plus petite égale à 1) est appelé *primitif*, et le polynôme irréductible dont il est racine est alors appelé un *polynôme primitif*. Tout corps $\mathbb{F}_q$ possède au moins un élément primitif.

§ A.5 Inverses sans table : l'algorithme d'Euclide étendu

Écrire une table complète des puissances, comme ci-dessus, convient pour $\mathbb{F}_{16}$ mais devient impraticable à la main pour des corps plus grands — et pourtant, inverser un seul élément du corps est exactement ce que font les algorithmes de décodage des Chapitres 6 à 9 à presque chaque étape (la formule de Forney, l'équation clé, l'évaluation de $1/g(\gamma_i)$ pour un code de Goppa, etc.). L'alternative efficace, aussi bien dans $\mathbb{F}_p$ que dans $\mathbb{F}_p[X]/(f)$, est l'*algorithme d'Euclide étendu*, exécuté sur les entiers modulo p ou sur les polynômes modulo f .

Exemple A.6 (Inversion d'un élément de $\mathbb{F}_{13}$). Pour calculer 8^{-1} dans $\mathbb{F}_{13}$: on exécute l'algorithme d'Euclide sur $(13, 8)$: $13 = 1 \cdot 8 + 5$, $8 = 1 \cdot 5 + 3$, $5 = 1 \cdot 3 + 2$, $3 = 1 \cdot 2 + 1$, $2 = 2 \cdot 1$. Par remontée : $1 = 3 - 1 \cdot 2 = 3 - 1 \cdot (5 - 3) = 2 \cdot 3 - 5 = 2(8 - 5) - 5 = 2 \cdot 8 - 3 \cdot 5 = 2 \cdot 8 - 3(13 - 8) = 5 \cdot 8 - 3 \cdot 13$. En réduisant mod 13 : $5 \cdot 8 \equiv 1 \pmod{13}$, donc $8^{-1} = 5$ dans $\mathbb{F}_{13}$ (vérification : $8 \times 5 = 40 = 3 \times 13 + 1$).

Exemple A.7 (Inversion d'un élément de $\mathbb{F}_{16} = \mathbb{F}_2[X]/(X^4 + X + 1)$). Pour inverser $\alpha^3 + \alpha$ (c'est-à-dire le polynôme $X^3 + X$), on exécute Euclide sur $(X^4 + X + 1, X^3 + X)$ sur $\mathbb{F}_2$: $X^4 + X + 1 = X \cdot (X^3 + X) + (X^2 + X + 1)$; $X^3 + X = (X + 1) \cdot (X^2 + X + 1) + X$ (vérification : $(X + 1)(X^2 + X + 1) = X^3 + X^2 + X + X^2 + X + 1 = X^3 + 1$, donc le reste est $X^3 + X - (X^3 + 1) = X - 1 = X + 1$ sur $\mathbb{F}_2$ — à refaire soigneusement à titre d'exercice, c'est exactement le type de calcul manuel demandé par l'Exercice A.9 ci-après). La remontée comme ci-dessus exprime 1 sous forme de combinaison de $X^4 + X + 1$ et $X^3 + X$ à coefficients polynomiaux sur $\mathbb{F}_2$; la réduction modulo $X^4 + X + 1$ donne directement l'inverse, sans qu'aucune table de puissances ne soit nécessaire.

§ A.6 Sous-corps et morphisme de Frobenius

Proposition A.8 (Critère de sous-corps). $\mathbb{F}_{p^d}$ est (isomorphe à) un sous-corps de $\mathbb{F}_{p^e}$ si et seulement si $d \mid e$.

C'est pourquoi, par exemple, $\mathbb{F}_4$ est inclus dans $\mathbb{F}_{16}$ (puisque $2 \mid 4$) mais pas dans $\mathbb{F}_8$ (puisque $2 \nmid 3$) — un fait qu'il convient de vérifier avant de chercher un raccourci de type « sous-corps » dans un calcul.

Définition A.9. L'automorphisme de Frobenius $\phi : \mathbb{F}_{p^e} \rightarrow \mathbb{F}_{p^e}$, $\phi(x) := x^p$, est un automorphisme de corps (il est additif car $(x + y)^p = x^p + y^p$ en caractéristique p — le « rêve du débutant », valable précisément parce que tous les coefficients binomiaux intermédiaires $\binom{p}{i}$, $0 < i < p$, sont divisibles par p ; il est clairement multiplicatif).

Le morphisme de Frobenius et ses itérés $\phi, \phi^2, \dots, \phi^{e-1}$ sont exactement les *automorphismes de corps* de $\mathbb{F}_{p^e}$ fixant $\mathbb{F}_p$; c'est l'ombre algébrique du fait, utilisé silencieusement tout au long du Chapitre 6, que le polynôme minimal d'un élément $\beta \in \mathbb{F}_{p^e}$ sur $\mathbb{F}_p$ a pour ensemble de racines exactement l'ensemble des *conjugués* $\beta, \beta^p, \beta^{p^2}, \dots$ de β — ce qui est précisément la manière dont les classes q -cyclotomiques ont été définies.

§ A.7 Table de référence rapide

Les petits corps suivants apparaissent fréquemment tout au long du cours ; un polynôme générateur est donné pour chacun afin que le lecteur puisse reconstruire le corps (comme dans la Section A.2) en cas de besoin.

Corps	Caractéristique/degré	Un polynôme irréductible générateur
$\mathbb{F}_4$	2^2	$X^2 + X + 1$
$\mathbb{F}_8$	2^3	$X^3 + X + 1$
$\mathbb{F}_9$	3^2	$X^2 + 1$
$\mathbb{F}_{16}$	2^4	$X^4 + X + 1$ (primitif)
$\mathbb{F}_{25}$	5^2	$X^2 + 4X + 2$
$\mathbb{F}_{27}$	3^3	$X^3 + 2X + 1$
$\mathbb{F}_{32}$	2^5	$X^5 + X^2 + 1$ (primitif)
$\mathbb{F}_{256}$	2^8	$X^8 + X^4 + X^3 + X^2 + 1$ (utilisé, par ex., dans AES, et dans les codes Reed–Solomon sur des octets comme dans les QR codes)